Kinetic Modeling of Cell Signaling

Module 1: Reaction networks as dynamical systems

Jim Faeder

MSCBIO 2025 · Introduction to Bioinformatics Programming in Python

Fall 2026

Resources

- **BioNetGen** software, documentation, and tutorials: bionetgen.org
- **VS Code extension**: search the Extensions panel for *BioNetGen*
- **BNG Playground**: write and simulate models in your browser, no installation ruleworld.github.io/bngplayground
- **bngsim**: fast ODE and stochastic simulation from Python bngsim.readthedocs.io · `pip install bngsim bionetgen`
- **Course repository**: models, notebooks, slides and handouts github.com/jrfaeder/RBM_Intro_BioNetGen



Course repository



BNG Playground

The book

Rule-Based Modeling of Biological Systems: Theory and practice with BioNetGen is an open book that develops the material of this course in more depth, with every model, figure, and exercise.

ruleworld.github.io/bionetgen-book

For this module, read:

- Chapter 1: Reaction networks as dynamical systems
- Chapter 7: Analyzing models (extends the pulse-shape exercise)



The book

Additional reading

- J. R. Faeder, M. L. Blinov, and W. S. Hlavacek (2009). **Rule-based modeling of biochemical systems with BioNetGen.** *Methods Mol. Biol.* **500**, 113–167. doi:10.1007/978-1-59745-525-1_5
  A thorough introduction to BNGL and the BioNetGen software.
- J. A. P. Sekar and J. R. Faeder (2012). **Rule-based modeling of signal transduction: a primer.** *Methods Mol. Biol.* **880**, 139–218. doi:10.1007/978-1-61779-833-7_9
  A self-contained tutorial: building a receptor model step by step, model assumptions, compartments, and a library of example rules.
- L. A. Harris et al. (2016). **BioNetGen 2.2: advances in rule-based modeling.** *Bioinformatics* **32**, 3366–3368. Free full text (PMC)
  What version 2.2 added: rate laws beyond mass action, faster stochastic and hybrid simulation, and SBML import.

The two *Methods in Molecular Biology* chapters are available through the Pitt library: follow the DOI link on campus or after signing in to the library.

Goals for this module

By the end of this module you should be able to

1. describe a biochemical process as a set of **species** and **reactions**
2. write the rate of each reaction using **mass-action kinetics**
3. turn a reaction network into a system of **ordinary differential equations**
4. encode the same network in the **BioNetGen language** (BNGL), simulate it, and interpret the results
5. explain how a **negative feedback** loop shapes the dynamics of a signaling response

Part 1

Why model reaction networks?

Reaction networks describe how molecule populations change

A **reaction network model** consists of

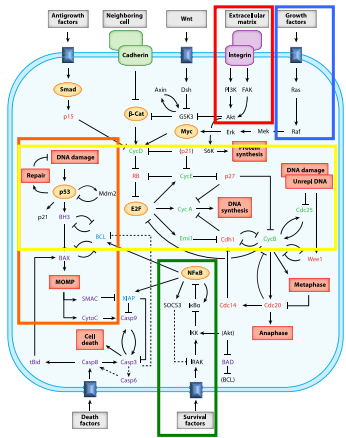
- **species**: the molecules or complexes we track
- **reactions**: how species are converted into one another
- **rate laws** and **rate constants**: how fast each reaction occurs
- **initial conditions**: the amount of each species at $t = 0$

The same framework describes atmospheric chemistry, epidemics, ecology, and the signaling and gene-regulatory networks that control cell behavior.

What a model gives us

- A precise statement of a mechanism
- Predictions of dynamics we cannot easily measure
- A way to test whether a mechanism can explain data
- Hypotheses for the next experiment

Toward an integrated model of the cell

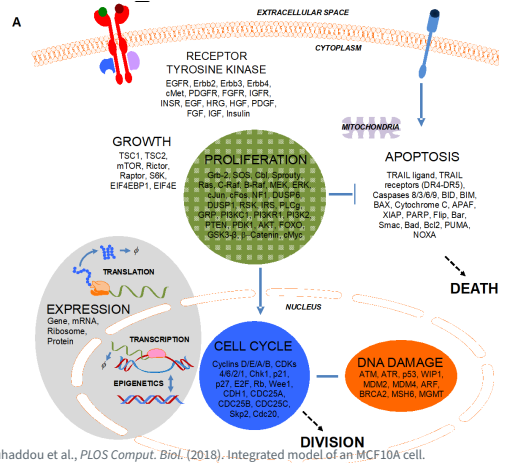


Tyson and Novak, *Annu. Rev. Phys. Chem.* **61**, 219–240 (2010)

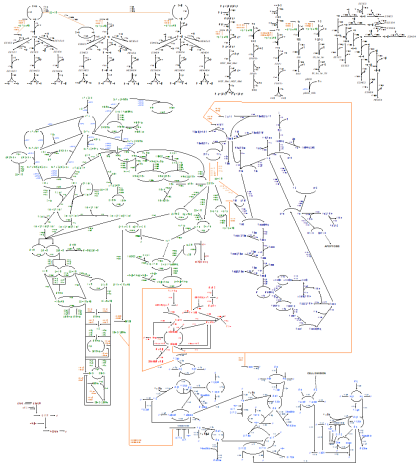
Cell fate is decided by many interacting signaling modules: growth-factor signaling, the cell cycle, DNA damage, apoptosis.

How do we link these modules together into models that predict how cells respond?

Large models are built from simple reaction steps



Bouhaddou et al., *PLOS Comput. Biol.* (2018). Integrated model of an MCF10A cell.

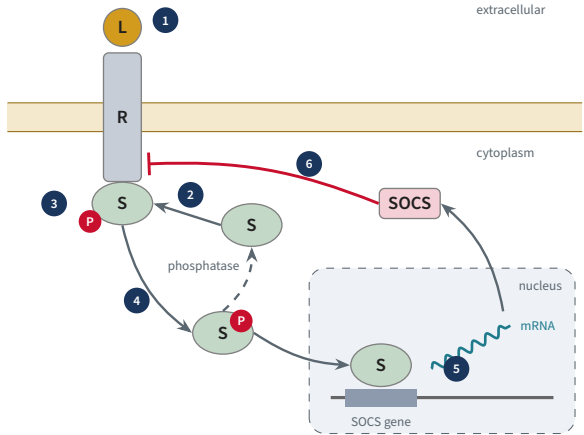


Part 2

A cytokine signaling circuit

Cytokines signal through receptors, STATs, and SOCS feedback

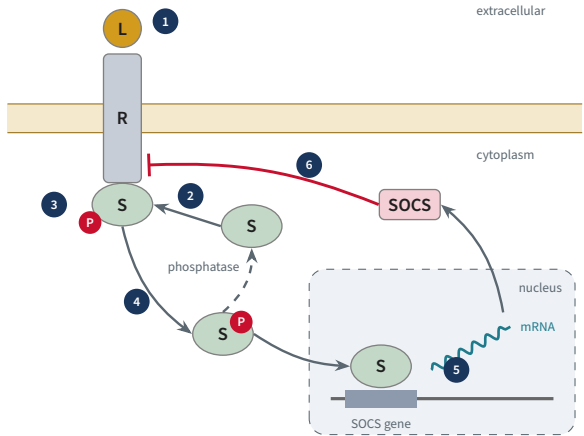
- **Cytokines** (e.g. interleukins, interferons) are secreted proteins that coordinate immune responses.
- Binding to their **receptor** activates receptor-associated JAK kinases.
- **STAT** transcription factors dock on the active receptor, are phosphorylated, and move to the nucleus.
- pSTAT induces genes including **SOCS** (suppressor of cytokine signaling).
- SOCS binds the receptor and **shuts off further STAT activation**: negative feedback.



We will build the model one reaction step at a time

1. Ligand–receptor binding
2. STAT binds the ligand-bound receptor
3. Receptor-bound STAT is activated
4. Activated STAT dissociates
5. pSTAT binds the SOCS promoter; SOCS is transcribed and translated
6. SOCS binds the receptor and blocks STAT binding

Each step adds reactions to the previous model. The files are `models/stat_step1_binding.bngl` through `stat_step6_feedback.bngl`.



Part 3

Step 1: Ligand–receptor binding

Binding is a reversible pair of reactions

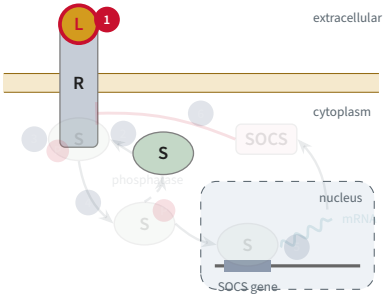
$$L + R \underset{k_{off}}{\overset{k_{on}}{\rightleftharpoons}} LR$$

Two elementary reactions, each with its own rate:

$$\text{binding:} \quad v_1 = k_{on}\,[L]\,[R]$$
$$\text{unbinding:} \quad v_2 = k_{off}\,[LR]$$

Law of mass action

The rate of an elementary reaction is proportional to the product of the concentrations of its reactants. The proportionality constant is the **rate constant**.



Units: k_{on} in $\mu M^{-1} min^{-1}$, k_{off} in min^{-1}

Each reaction contributes to the rate of change of its species

	v_1 (bind)	v_2 (unbind)
L	-1	$+1$
R	-1	$+1$
LR	$+1$	-1

Reactants are consumed ($-$), products are created ($+$).

$$\frac{d[\mathrm{L}]}{dt} = -k_{\mathrm{on}}[\mathrm{L}][\mathrm{R}] + k_{\mathrm{off}}[\mathrm{LR}]$$

$$\frac{d[\mathrm{R}]}{dt} = -k_{\mathrm{on}}[\mathrm{L}][\mathrm{R}] + k_{\mathrm{off}}[\mathrm{LR}]$$

$$\frac{d[\mathrm{LR}]}{dt} = +k_{\mathrm{on}}[\mathrm{L}][\mathrm{R}] - k_{\mathrm{off}}[\mathrm{LR}]$$

Ordinary differential equations (ODEs) follow mechanically from the list of reactions and their rate laws.

The stoichiometry matrix gives a compact general form

Collect the concentrations in a vector $\mathbf{X}$ and the reaction rates in $\mathbf{v}$:

$$\frac{d\mathbf{X}}{dt} = \mathbf{S}\,\mathbf{v}(\mathbf{X}), \qquad \frac{d}{dt}\begin{pmatrix}[\mathrm{L}]\\ [\mathrm{R}]\\ [\mathrm{LR}]\end{pmatrix} = \underbrace{\begin{pmatrix}-1 & +1\\ -1 & +1\\ +1 & -1\end{pmatrix}}_{\mathbf{S}} \underbrace{\begin{pmatrix}k_{\mathrm{on}}[\mathrm{L}][\mathrm{R}]\\ k_{\mathrm{off}}[\mathrm{LR}]\end{pmatrix}}_{\mathbf{v}(\mathbf{X})}$$

S_{ij} = (times species i is a product of reaction j) $-$ (times species i is a reactant of reaction j)

- **S** has one row per species and one column per reaction.
- This form holds for *any* reaction network; only **S** and **v** change.

Conservation laws and equilibrium

Conservation. Binding only moves molecules between free and bound forms:

$$[\mathrm{L}] + [\mathrm{LR}] = L_0$$
$$[\mathrm{R}] + [\mathrm{LR}] = R_0$$

so only one of the three ODEs is independent.

These appear as linear dependencies among the rows of **S**.

Equilibrium. At steady state $v_1 = v_2$:

$$\frac{[\mathrm{L}][\mathrm{R}]}{[\mathrm{LR}]} = \frac{k_{\mathrm{off}}}{k_{\mathrm{on}}} \equiv K_D$$

Substituting the conservation laws gives a quadratic for [LR]:

$$[\mathrm{LR}]_{\mathrm{eq}} = \tfrac{1}{2}\left[\, b - \sqrt{b^2 - 4L_0R_0}\,\right],$$

$$b = L_0 + R_0 + K_D$$

Small models can be solved by hand. Real signaling networks cannot, so we solve the ODEs numerically.

ODEs for reaction networks are solved numerically

Simplest idea (forward Euler): take small time steps Δt,

$$\mathbf{X}(t + \Delta t) \approx \mathbf{X}(t) + \Delta t\ \mathbf{S}\,\mathbf{v}(\mathbf{X}(t))$$

In practice:

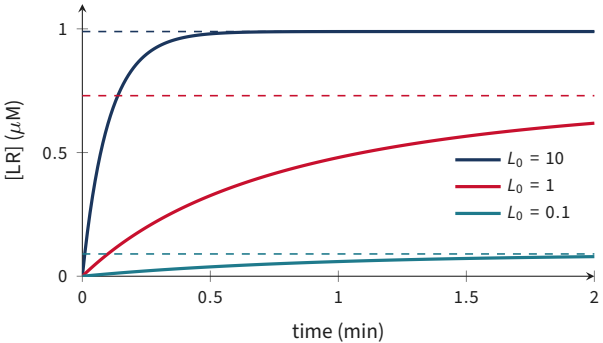
- The equations are **nonlinear** and usually have no closed-form solution.
- Reactions occur on very different time scales, which makes the system **stiff**: simple methods need impractically small steps.
- Production solvers (CVODE, LSODA) adapt the step size and use implicit methods for stiff systems.

In BioNetGen

`simulate({method=>"ode",...})` uses CVODE.

In Python, `bngsim.Simulator(model, method="ode")` runs the same model with CVODE.

Binding approaches equilibrium faster and further at higher ligand



$R_0 = 1\ \mu\text{M}$, $k_{\text{on}} = 1\ \mu\text{M}^{-1}\text{min}^{-1}$, $k_{\text{off}} = 0.1\ \text{min}^{-1}$ ($K_D = 0.1\ \mu\text{M}$).

Dashed lines: $[\text{LR}]_{\text{eq}}$ from the formula on the previous slide.

The relaxation rate is $k_{\text{on}}([\text{L}] + [\text{R}]) + k_{\text{off}}$ near equilibrium, so higher ligand also means a faster response.

The same model in BNGL

```
begin model

begin parameters
  L0        10      # ligand (uM)
  R0        1       # receptor (uM)
  kon       1       # binding (1/uM/min)
  koff      0.1     # unbinding (1/min)
end parameters

begin molecule types
  L
  R
  LR
end molecule types

begin species
  L    L0
  R    R0
end species
```

```
begin observables
  Molecules Lfree   L
  Molecules Rfree   R
  Molecules LR      LR
end observables

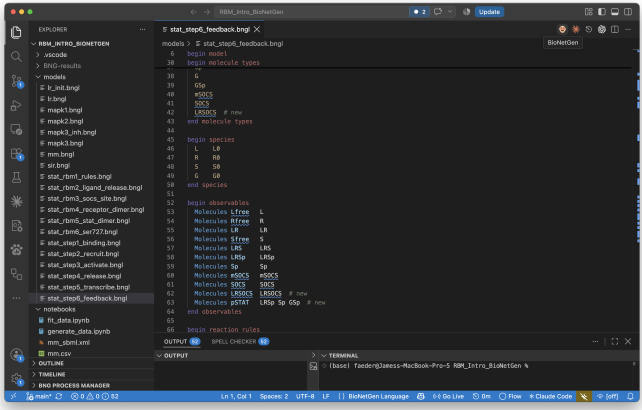
begin reaction rules
  # Ligand binds receptor
  LRbind: L + R <-> LR kon, koff
end reaction rules

end model

generate_network({overwrite=>1})
simulate({method=>"ode",t_end=>2,n_steps=>200})
```

- **observables**: quantities to output
- **reaction rules**: `<->` is reversible; forward and reverse rate constants
- **actions** after `end model`: generate the network, then simulate

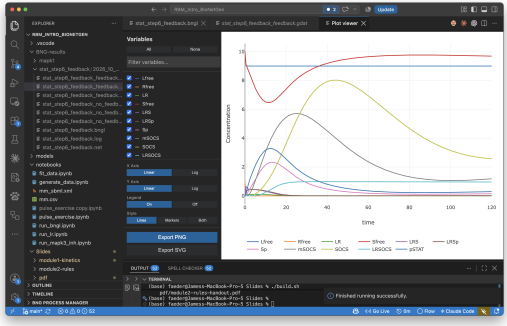
Running a model in VS Code



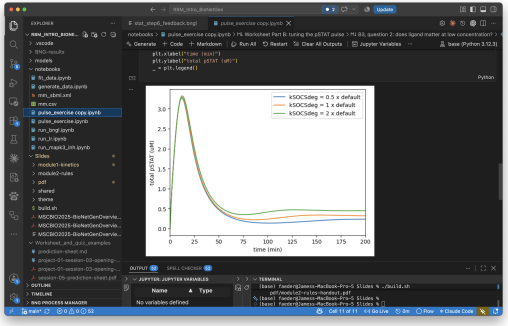
VS Code extension: Ali Sinan Saglam

- Open a `.bngl` file
- Click the **run** button in the editor toolbar
- Results go to `BNG-results/<model>/<timestamp>/`

Interactive plotting and Python notebooks

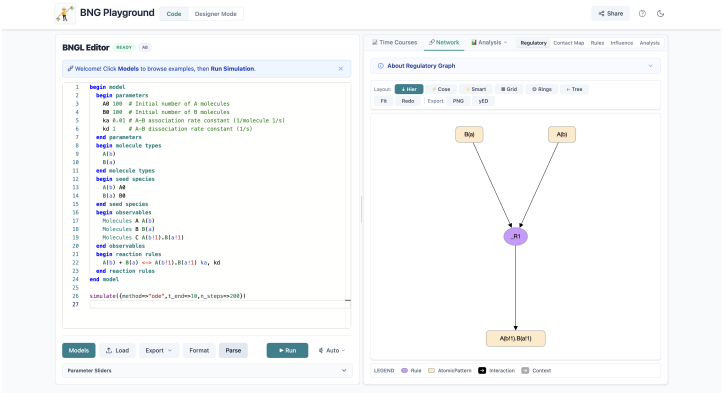


Built-in plotting of `.gdat` output



Deeper analysis in Jupyter with bngsim

BNG Playground: BioNetGen in your browser



BNGL editor (left) and the regulatory graph of a binding rule (right)

- No installation: write, simulate, and analyze models
- Built-in examples
- Share a model as a link
- Paste in any file from `models/`

ruleworld.github.io/bngplayground

Part 4

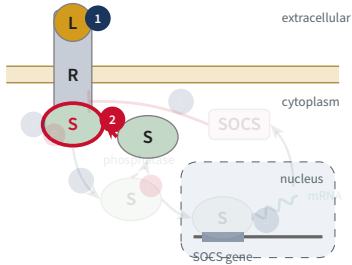
Building the signaling circuit in BNGL

Step 2: STAT binds the ligand-bound receptor

$$LR + S \rightleftharpoons LRS$$

New lines in `stat_step2_recruit.bngl`:

```
begin parameters
  S0     10   # STAT (uM)
  kSon   1    # STAT binding (1/uM/min)
  kSoff  1    # STAT unbinding (1/min)
end parameters
begin molecule types
  S
  LRS
end molecule types
begin species
  S  S0
end species
begin reaction rules
  Sbind: LR + S <-> LRS kSon, kSoff
end reaction rules
```



STAT can bind *only* the ligand-bound receptor LR, never free R. This is how ligand binding **enables** STAT recruitment.

Every new species needs a molecule type; species present at $t = 0$ also need an initial amount.

Step 3: receptor-bound STAT is activated

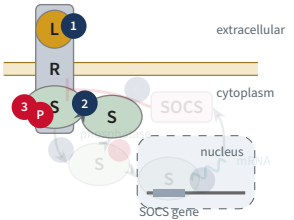
$$\mathrm{LRS} \xrightarrow{k_{\mathrm{act}}} \mathrm{LRS_p}$$

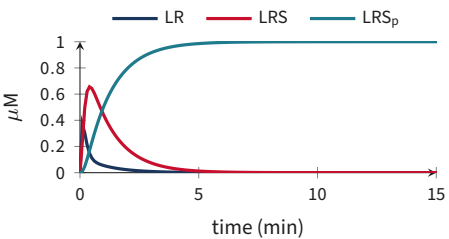
A receptor-associated kinase (JAK) phosphorylates the docked STAT: a **first-order**, irreversible reaction with $v = k_{\mathrm{act}}[\mathrm{LRS}]$.

```
kact  1   # STAT activation (1/min)
LRSp
Sact: LRS -> LRSp kact
```

one new line each in parameters, molecule types, and reaction rules

Problem: pSTAT never leaves the receptor. Every receptor ends up stuck as $\mathrm{LRS_p}$.





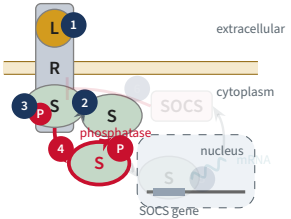
Step 4: activated STAT dissociates from the receptor

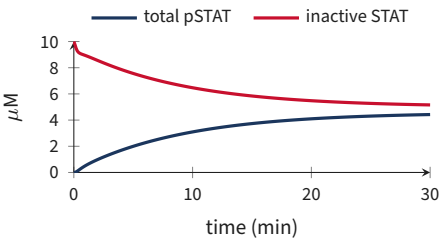
$$LRS_p \xrightarrow{k_{diss}} LR + S_p \qquad S_p \xrightarrow{k_{dephos}} S$$

Releasing pSTAT frees the receptor to activate another STAT. A phosphatase returns free pSTAT to the inactive pool.

```
kdiss    1    # pSTAT release (1/min)
kdephos  0.1  # dephosphorylation (1/min)
Sp
Srelease: LRSp -> LR + Sp kdiss
Sdephos:  Sp -> S kdephos
```

pSTAT now reaches a steady state set by the balance of activation and dephosphorylation.



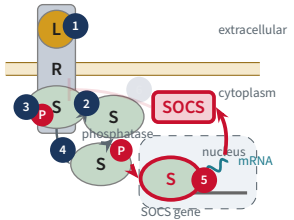


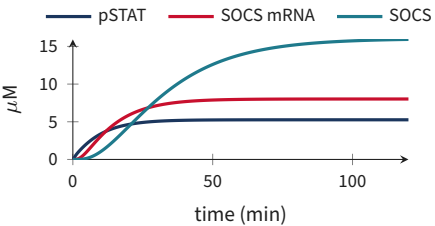
Step 5: pSTAT induces SOCS expression

```
# pSTAT binds the SOCS promoter
Gbind: Sp + G <-> GSp kGon, kGoff
# Bound promoter is transcribed
Transcribe: GSp -> GSp + mSOCS ktx
# mRNA degradation
mRNAdeg: mSOCS -> 0 kmdeg
# Translation
Translate: mSOCS -> mSOCS + SOCS ktl
# SOCS protein degradation
SOCSdeg: SOCS -> 0 kSOCSdeg
```

- **Synthesis**: a species on both sides (`GSp`, `mSOCS`) is a template and is not consumed.
- **Degradation**: `0` means nothing; the molecule is removed.

SOCS accumulates, but it has no target yet, so pSTAT is unaffected.





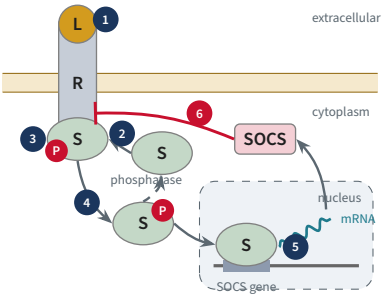
Step 6: SOCS binds the receptor and blocks STAT recruitment

$$\mathrm{LR} + \mathrm{SOCS} \rightleftharpoons \mathrm{LR{\cdot}SOCS}$$

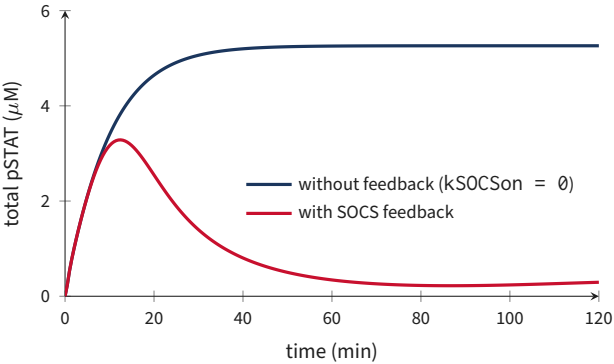
SOCS competes with STAT for the active receptor. LR·SOCS cannot recruit STAT, so pSTAT production drops. The loop is closed:

$$\mathrm{pSTAT} \rightarrow \mathrm{SOCS} \dashv \mathrm{receptor} \rightarrow \mathrm{pSTAT}$$

```
kSOCSon   1     # (1/uM/min)
kSOCSoff  0.01  # (1/min)
LRSOCS
SOCSbind: LR + SOCS <-> LRSOCS kSOCSon, kSOCSoff
```



Negative feedback turns a sustained signal into a pulse



- Early response is identical: SOCS has not been made yet.
- As SOCS accumulates it sequesters receptors, and pSTAT falls to a low level despite continued ligand.
- The delay comes from transcription and translation.

Both curves come from `stat_step6_feedback.bngl`, which runs the model twice using `setParameter`.

The complete network

	reaction	rate
1	$L + R \rightleftharpoons LR$	k_{on}, k_{off}
2	$LR + S \rightleftharpoons LRS$	k_{Son}, k_{Soff}
3	$LRS \rightarrow LRS_p$	k_{act}
4	$LRS_p \rightarrow LR + S_p$	k_{diss}
	$S_p \rightarrow S$	k_{dephos}
5	$S_p + G \rightleftharpoons GS_p$	k_{Gon}, k_{Goff}
	$GS_p \rightarrow GS_p + mSOCS$	k_{tx}
	$mSOCS \rightarrow 0$	k_{mdeg}
	$mSOCS \rightarrow mSOCS + SOCS$	k_{tl}
	$SOCS \rightarrow 0$	$k_{SOCSdeg}$
6	$LR + SOCS \rightleftharpoons LR{\cdot}SOCS$	$k_{SOCSon}, k_{SOCSoff}$

- 11 lines of BNGL generate **15 reactions** (4 are reversible) among **12 species**.
- **S** is a 12×15 matrix; BioNetGen builds it and the ODEs for you (see the `.net` file).
- Every species had to be named by hand: L, LR, LRS, LRS_p, LR·SOCS, …

Part 5

Looking ahead

Naming every complex by hand does not scale

Our model made strong simplifying assumptions. Relax a few of them:

- SOCS can also bind receptors that already carry STAT (LRS, LRS_p)
- Ligand can dissociate from *any* receptor complex, not only LR
- The receptor has two chains, each recruiting its own STAT
- STATs dimerize before entering the nucleus

Each feature **multiplies** the number of distinct complexes, and each complex needs its own species name and its own copy of every reaction it takes part in.

Module 2: represent molecules with internal structure (binding sites and states) and write rules that generate the network automatically.

ODEs describe average behavior

- ODEs treat concentrations as continuous and deterministic.
- In a single cell, some species are present in very small numbers: one or two copies of a gene, a few mRNA molecules.
- Then random timing of individual reactions matters, and cells with identical parameters respond differently.
- The **same reaction network** can be simulated stochastically with the Gillespie algorithm: `simulate({method=>"ssa",...})`.

We return to stochastic simulation in a later module.

End-of-class exercises

On the worksheet (`Worksheets/module1-exercises.pdf`):

A. Check the ligand–receptor equilibrium against the formula.

B. **Tune the pSTAT pulse**: predict, then test, how six parameters change the time of the peak, its height, and the adapted level.

C. Remove the phosphatase. What sets the final level of pSTAT?

D. Let SOCS bind STAT-loaded receptors. What new species and reactions do you need?

No local setup yet? Paste any model from `models/` into BNG Playground (`ruleworld.github.io/bngplayground`) and run it in your browser.

Summary

- A reaction network is defined by its species, reactions, rate laws, and initial conditions.
- Mass-action rate laws turn the network into ODEs: $d\mathbf{X}/dt = \mathbf{S}\,\mathbf{v}(\mathbf{X})$.
- BNGL describes the network directly; BioNetGen generates and integrates the ODEs.
- Building the cytokine–STAT–SOCS circuit one step at a time shows how each mechanism contributes; negative feedback produces an adaptive pulse of pSTAT.
- Writing every complex as a separate species quickly becomes unmanageable. This motivates rule-based modeling, the subject of Module 2.